

```

import pandas as pd
import re
import nltk
from bs4 import BeautifulSoup
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import os

# 1. Download necessary NLP dictionaries
print("Downloading NLTK data...")
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('omw-1.4', quiet=True)

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

# Pointing exactly to Colab's main folder where your files are right now!
DATA_DIR = '/content/'

def load_and_unify_data():
    all_data = []

    # Load SMS Spam
    try:
        sms_path = os.path.join(DATA_DIR, 'SMSSpamCollection.csv')
        sms_df = pd.read_csv(sms_path, encoding='latin-1')
        sms_df = sms_df[['v1', 'v2']].rename(columns={'v1': 'label', 'v2': 'text'})
        sms_df['label'] = sms_df['label'].map({'ham': 0, 'spam': 1})
        all_data.append(sms_df)
        print("✅ Loaded SMSSpamCollection")
    except Exception as e:
        print(f"❌ Error with SMS data: {e}")

    # Load Email Datasets
    email_files = ['CEAS_08.csv', 'Enron.csv', 'Ling.csv', 'Nazario.csv', 'Nigerian_Fraud.csv', 'SpamAssasin.csv', 'phishing']

    for file in email_files:
        try:
            df = pd.read_csv(os.path.join(DATA_DIR, file))
            # Standardize columns: grab text and label
            if 'body' in df.columns and 'label' in df.columns:
                df = df[['body', 'label']].rename(columns={'body': 'text'})
            elif 'text' in df.columns and 'label' in df.columns:
                df = df[['text', 'label']]
            else:
                df = df.iloc[:, [0, 1]]
                df.columns = ['text', 'label']

            df['label'] = df['label'].astype(int)
            all_data.append(df)
            print(f"✅ Loaded {file}")
        except Exception as e:
            print(f"❌ Error with {file}: {e}")

    # Combine everything into one massive dataset
    combined_df = pd.concat(all_data, ignore_index=True)
    combined_df.dropna(subset=['text'], inplace=True)
    combined_df.drop_duplicates(subset=['text'], inplace=True)

    return combined_df

# The NLP Cleaning Function
def clean_text(text):
    if not isinstance(text, str):
        return ""

    # Try to parse HTML, fallback to raw text if it fails
    try:
        text = BeautifulSoup(text, "html.parser").get_text()
    except:
        pass

    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '<URL>', text, flags=re.MULTILINE)
    text = re.sub(r'\S+@\S+', '<EMAIL>', text)
    text = re.sub(r'^a-zA-Z<\s', ' ', text)

    words = text.split()
    cleaned_words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]
    return " ".join(cleaned_words)

```

```

# Execute the pipeline
print("\n--- Starting Data Unification ---")
df = load_and_unify_data()
print(f"Total rows before cleaning: {len(df)}")

print("\n--- Applying Text Cleaning ---")
print("(This will take a few minutes. Let it run!)")
df['clean_text'] = df['text'].apply(clean_text)

df = df[df['clean_text'].str.strip() != '']
print(f"Total rows after cleaning: {len(df)}")

# Save the final preprocessed dataset right there in Colab
save_path = '/content/omniguard_preprocessed_text.csv'
df.to_csv(save_path, index=False)
print(f"\n🎉 Preprocessing complete! Clean data saved to: {save_path}")

```

Downloading NLTK data...

--- Starting Data Unification ---

✗ Error with SMS data: "None of [Index(['v1', 'v2'], dtype='object')] are in the [columns]"

✓ Loaded CEAS_08.csv

✓ Loaded Enron.csv

✓ Loaded Ling.csv

✓ Loaded Nazario.csv

✓ Loaded Nigerian_Fraud.csv

✓ Loaded SpamAssasin.csv

✗ Error with phishing_email.csv: Cannot convert non-finite values (NA or inf) to integer

Total rows before cleaning: 82483

--- Applying Text Cleaning ---

(This will take a few minutes. Let it run!)

/tmp/ipykernel_6711/698037441.py:70: MarkupResemblesLocatorWarning: The input passed in on this line looks more like a URL t

If you meant to use BeautifulSoup to parse the web page found at a certain URL, then something has gone wrong. You should u

However, if you want to parse some data that happens to look like a URL, then nothing has gone wrong: you are using Beautifl

```

from bs4 import MarkupResemblesLocatorWarning
import warnings

```

```
warnings.filterwarnings("ignore", category=MarkupResemblesLocatorWarning)
```

```
text = BeautifulSoup(text, "html.parser").get_text()
```

Total rows after cleaning: 82451

🎉 Preprocessing complete! Clean data saved to: /content/omniguard_preprocessed_text.csv

Start coding or [generate](#) with AI.

```

from google.colab import files
files.download('/content/omniguard_preprocessed_text.csv')

```

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 1. Load the clean data we just saved
DATA_PATH = '/content/omniguard_preprocessed_text.csv'
print("Loading clean data...")
df = pd.read_csv(DATA_PATH)

# Drop any accidental blank rows
df = df.dropna(subset=['clean_text', 'label'])

# 2. Split into Features (X) and Target (y)
X = df['clean_text']
y = df['label']

# 3. Train/Test Split (80% of data to train the AI, 20% to test it)
print("Splitting data into 80% Training and 20% Testing sets...")
X_train_text, X_test_text, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Vectorize the text (Convert words into math!)

```

```

print("Converting words into numerical data (TF-IDF)...")
vectorizer = TfidfVectorizer(max_features=5000)
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)

# 5. Train the Logistic Regression Model
print("Training the Logistic Regression AI... (Give it a few seconds)")
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train, y_train)

# 6. Make Predictions on the hidden test data
print("Generating predictions...")
y_pred = log_model.predict(X_test)

# 7. Evaluate how well it did
print("\n" + "="*40)
print("👮 OMNIGUARD MODEL RESULTS 🏆")
print("="*40)
print(f"Overall Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%\n")
print("Detailed Classification Report:")
print(classification_report(y_test, y_pred, target_names=['Safe (0)', 'Fraud (1)']))

# 8. Draw a Confusion Matrix to visualize the AI's brain
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicted Safe', 'Predicted Fraud'],
            yticklabels=['Actual Safe', 'Actual Fraud'])
plt.title('OmniGuard: How Many Scams Did We Catch?')
plt.show()

```

Loading clean data...
 Splitting data into 80% Training and 20% Testing sets...
 Converting words into numerical data (TF-IDF)...
 Training the Logistic Regression AI... (Give it a few seconds)
 Generating predictions...

=====
 👮 OMNIGUARD MODEL RESULTS 🏆
 =====
 Overall Accuracy: 97.56%

Detailed Classification Report:

	precision	recall	f1-score	support
Safe (0)	0.98	0.97	0.97	7921
Fraud (1)	0.97	0.98	0.98	8570
accuracy			0.98	16491
macro avg	0.98	0.98	0.98	16491
weighted avg	0.98	0.98	0.98	16491



```

# 1. Write your own custom messages to test!
custom_messages = [
    "Hey, are we still on for the project meeting tomorrow?",

```